Benchmark Summary for `aerosolDynamicsFoam`

1 Purpose

The benchmarks were performed with the `testImpactor` case to evaluate three implementation choices:

1. whether `analyticPositionTracking` permits larger particle time steps without losing the relevant particle fate information;
2. whether the replicated-mesh particle-parallel mode scales well for a fixed carrier-flow, one-way Lagrangian calculation;
3. how the event-driven and ordinary time-loop modes compare.

The main result is that analytic position tracking makes coarse particle time steps considerably more usable. In the impaction case, it reduces the coarse-step bias in the outlet/substrate split, so the calculation can use a larger `deltaTMax` while preserving the practically important particle fate trends. Full convergence of the fate counts was still obtained only when the particle step was reduced to about $\Delta t_{\max} = 10^{-7}\,\mathrm{s}$ for the present case, but the analytic displacement update gives a much better large-step approximation than the legacy position update.

2 Benchmark Set-up

All benchmark cases were derived from `testImpactor`. The original case was copied to a temporary benchmark directory, and the carrier-flow field and mesh were kept fixed. The particle size range was 0.10–1.00 µm, with a spacing of 0.05 µm. The injection positions were duplicated from the original valid injection set, giving 200 parcels per size bin and 3800 parcels in total.

The event-driven calculations used

$$t_{\mathrm{track}} = 0.009119\,\mathrm{s}.$$

The particle-parallel calculations were run with four MPI ranks using `-particleParallel`. This is not OpenFOAM mesh-decomposition parallelism: each rank reads the full mesh and carrier field, while the injected parcels are split into particle shards.

3 Effect of `analyticPositionTracking`

Table 1 shows the event-driven results with `analyticPositionTracking true`. The finest case, $\Delta t_{\max} = 10^{-8}\,\mathrm{s}$, is used here as the reference.

At coarse steps, the largest fate-distribution differences relative to the $10^{-8}\,\mathrm{s}$ reference occurred on the small-particle side. Using the total-variation difference of the patch fate distribution per size bin, the affected size bins were:

Table 1: Event-driven benchmark with `analyticPositionTracking true`.

$\Delta t_{\max}$	Wall time [s]	outlet	substrate	firstConv	firstNozzle	secondConv
10^{-4}	7.68	1266	2420	76	38	0
10^{-5}	7.65	1266	2420	76	38	0
10^{-6}	7.36	1260	2426	76	6	16
10^{-7}	20.06	1316	2370	76	8	30
10^{-8}	189.82	1290	2396	76	8	30

- 10^{-4} and 10^{-5} s: mainly 0.10–0.25 µm, with a maximum difference of about 8%;
- 10^{-6} s: mainly 0.10–0.25 µm, with an additional sensitivity around 0.85 µm;
- 10^{-7} s: no size bin exceeded a 5% difference, and the maximum difference was about 2%.

Table 2 shows the same event-driven benchmark with `analyticPositionTracking false`. The same `analyticPositionTracking true`, 10^{-8} s case is used as the reference for the error statements.

Table 2: Event-driven benchmark with `analyticPositionTracking false`.

$\Delta t_{\max}$	Wall time [s]	outlet	substrate	firstConv	firstNozzle	secondConv	secondNozzle
10^{-4}	1.23	1184	2502	82	32	0	0
10^{-5}	1.24	1184	2502	82	32	0	0
10^{-6}	1.51	1180	2506	76	8	20	10
10^{-7}	9.18	1288	2398	76	8	30	0

Without analytic position tracking, the coarse-step solution over-predicts substrate collection and under-predicts outlet escape. For example, at $\Delta t_{\max} = 10^{-4}$ s, the analytic update gives 1266 outlet parcels, whereas the legacy position update gives only 1184 outlet parcels; the 10^{-8} s reference gives 1290. Thus the analytic displacement update substantially reduces the coarse-step bias in the primary outlet/substrate split.

The `false` runs also converge when the time step is reduced. At 10^{-7} s, the total fate counts are very close to the reference: 1288 outlet parcels and 2398 substrate parcels, compared with 1290 and 2396 in the reference. However, the benefit of `analyticPositionTracking` is that useful accuracy is retained at a larger particle time step.

4 Comparison with `icoUncoupledKinematicParcelFoam`

As a validation check, the same impactor benchmark was also run with the standard OpenFOAM `icoUncoupledKinematicParcelFoam` solver. This solver does not contain the new event-driven tracking, analytic displacement update, or particle-sharding options. It is therefore a useful baseline for checking that the present implementation recovers the legacy behavior when `analyticPositionTracking` is disabled.

Table 3 compares the custom solver with `analyticPositionTracking false` against `icoUncoupledKinematicl`
The fate tuple is

$$(N_{\text{outlet}}, N_{\text{substrate}}, N_{\text{firstConv}}, N_{\text{firstNozzle}}, N_{\text{secondConv}}).$$

Table 3: Legacy-behavior validation against `icoUncoupledKinematicParcelFoam`.

Δt	Custom solver, analytic=false	`icoUncoupled`	Custom time [s]	`ico` time [s]
10^{-4}	(1184, 2502, 82, 32, 0)	(1184, 2502, 82, 32, 0)	1.23	3.82
10^{-5}	(1184, 2502, 82, 32, 0)	(1186, 2500, 80, 34, 0)	1.24	4.36
10^{-6}	(1180, 2506, 76, 8, 20)	(1208, 2478, 76, 8, 30)	1.51	10.43
10^{-7}	(1288, 2398, 76, 8, 30)	(1288, 2398, 76, 8, 30)	9.18	96.47

The agreement is strongest at the finest tested step: $\Delta t = 10^{-7}$ s gives identical patch fate counts between the custom solver with `analyticPositionTracking false` and `icoUncoupledKinematicParcelFoam`. At coarser steps, the two solvers show the same qualitative trend, with small differences that decrease as the time step is refined. This is an important regression test: disabling the new analytic displacement update recovers the established solver behavior. The remaining improvement with `analyticPositionTracking true` can therefore be attributed to the new position update rather than to an unrelated change in the particle model or impactor set-up.

5 Event-driven Versus Ordinary Time Loop

The ordinary non-event-driven time-loop calculations were also run with four particle shards. In this mode the carrier time step controls the number of outer time-loop iterations. The measured results are shown in Table 4.

Table 4: Ordinary time-loop benchmark, without event-driven tracking.

Δt	Wall time [s]	outlet	substrate	firstConv	firstNozzle	secondConv
10^{-4}	7.63	1266	2420	76	38	0
10^{-5}	7.67	1268	2418	76	38	0
10^{-6}	6.48	1290	2396	76	6	32
10^{-7}	55.70	1308	2378	76	8	30

The absolute timings include MPI start-up and I/O/logging overhead, so they should not be over-interpreted as a smooth time-step scaling curve. The important algorithmic point is that the ordinary time loop requires many more outer loop iterations as Δt decreases. In contrast, event-driven tracking advances the particles over a prescribed physical tracking horizon without repeatedly cycling the fixed Eulerian field.

6 Particle Parallelism

The solver uses a replicated-mesh particle-sharding strategy. Each rank reads the full mesh and carrier fields, and only the particles are divided among ranks. This is well matched to the present problem because the carrier flow is fixed and the calculation is one-way coupled.

A two-shard benchmark on the impactor case gave the following timings:

If the two shards are executed concurrently, the expected speed-up is roughly

$$\frac{7.45}{3.75} \approx 1.99,$$

Table 5: Two-shard particle-parallel timing check.

Run	Wall time [s]	Relative work
Serial	7.45	all parcels
Shard 0	3.73	half the parcels
Shard 1	3.75	half the parcels

which is essentially ideal two-way scaling for this particle-dominated benchmark. A small-parcel-count MPI test showed much less benefit because the fixed costs of MPI launch, mesh reading, and output become comparable to the particle work.

Table 6: Standard OpenFOAM cell-decomposition timing with `icoUncoupledKinematicParcelFoam`.

Δt	Serial time [s]	Four-way mesh-decomposed time [s]
10^{-4}	3.82	4.01
10^{-5}	4.36	4.73
10^{-6}	10.43	10.49
10^{-7}	96.47	97.24

Table 6 shows that standard cell decomposition did not accelerate the baseline `icoUncoupledKinematicParcelF` benchmark. For this fixed-flow particle-tracking problem, the mesh-decomposition overhead and particle migration costs do not pay for themselves. This contrasts with particle sharding, which directly divides the dominant particle work.

7 Why Particle Sharding Is Preferable Here

Standard cell decomposition is generally not the best first choice for this solver. The carrier field is fixed, and the expensive part is tracking many independent particles. Decomposing cells introduces processor patches and particle migration between subdomains, but it does not remove the main particle-tracking cost. It can also create load imbalance when many particles collect or escape in some parts of the geometry but not others.

Particle sharding avoids those costs. Since each rank has the full mesh, particles do not need to communicate when crossing a processor boundary. For one-way tracking on a fixed flow field, this is a natural decomposition of the work.

8 Caveats

Both approaches have caveats.

Replicated-mesh particle sharding. Memory usage grows approximately with the number of ranks because every rank holds the full mesh and carrier fields. The output is also sharded, so post-processing must combine `kinematicCloud_shard*` results. The method is most effective when the parcel count is large enough that particle tracking dominates the fixed cost of reading the case and writing output. Load balance is based on the initial particle shard assignment, so cases with strongly size-dependent or position-dependent early deposition can still become imbalanced.

Cell-decomposition parallelism. Cell decomposition can be useful for a fully coupled Eulerian–Lagrangian solver or for an expensive Eulerian flow solve. It is less attractive here because the carrier field is not solved and the particles are the dominant work. Cell decomposition also requires processor-boundary particle migration and standard reconstruction workflows, which add overhead for this fixed-flow one-way tracking use case.

9 Conclusion

For the impactor benchmark, `analyticPositionTracking` improves the coarse-time-step behavior of the particle fate prediction. It reduces the large-step outlet/substrate bias and allows larger particle steps to be used with less loss of accuracy. The most sensitive size range in the present case is the small-particle side, especially 0.10–0.40 µm, with an additional localized sensitivity around 0.85 µm in some runs.

The comparison with `icoUncoupledKinematicParcelFoam` supports the correctness of the implementation. When `analyticPositionTracking` is disabled, the custom solver reproduces the legacy solver's fine-step patch fate counts. The analytic-position branch then improves the coarse-step behavior without changing the underlying particle model.

For production calculations of this type, replicated-mesh particle sharding is a strong parallelization strategy. It directly divides the particle work and avoids processor-patch particle migration. The trade-off is increased memory usage and sharded post-processing output.